

WEEK 5 PRACTICAL

QUESTION 1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Aim

To implement communication between a parent and child process using an anonymous pipe and measure the communication time.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define COUNT 100000

int main() {
    int fd[2];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        // Child - Consumer
        close(fd[1]);

        int data;
        int received = 0;

        while (read(fd[0], &data, sizeof(data)) > 0) {
            received++;
        }

        close(fd[0]);
    }
```

```

        printf("Consumer received: %d values\n", received);
    }
    else {
        // Parent - Producer
        close(fd[0]);

        int data;
        struct timespec start, end;

        clock_gettime(CLOCK_MONOTONIC, &start);

        for (int i = 0; i < COUNT; i++) {
            data = i;
            write(fd[1], &data, sizeof(data));
        }

        close(fd[1]);

        clock_gettime(CLOCK_MONOTONIC, &end);

        double time_taken =
            (end.tv_sec - start.tv_sec) +
            (end.tv_nsec - start.tv_nsec) / 1e9;

        printf("Producer generated: %d values\n", COUNT);
        printf("Communication time: %.6f seconds\n", time_taken);

        wait(NULL);
    }

    return 0;
}

```

Output

```

chakrika@blue:~/OSSP$ mkdir -p Week5
chakrika@blue:~/OSSP$ cd Week5
chakrika@blue:~/OSSP/Week5$ pwd
/home/chakrika/OSSP/Week5
chakrika@blue:~/OSSP/Week5$ nano producer_consumer.c
chakrika@blue:~/OSSP/Week5$ gcc producer_consumer.c -o producer_consumer
chakrika@blue:~/OSSP/Week5$ ./producer_consumer
Producer generated: 100000 values
Communication time: 0.076285 seconds
Consumer received: 100000 values

```

Observation

The parent process generated data and sent it through the pipe. The child process successfully received the data, and the communication time was measured.

Result

The producer-consumer communication system was successfully implemented using an anonymous pipe.

QUESTION 2

Develop a program that executes the equivalent of the shell command `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Aim

To implement a pipeline between two commands using `fork()`, `pipe()`, `dup2()` and `exec()`.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid1 = fork();

    if (pid1 == 0) {
        // First child: ls -l
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("exec ls failed");
        exit(1);
    }

    pid_t pid2 = fork();
```

```

    if (pid2 == 0) {
        // Second child: grep ".c"
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("exec grep failed");
        exit(1);
    }

    // Parent
    close(fd[0]);
    close(fd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Command pipeline completed.\n");

    return 0;
}

```

Output

```

chakrika@blue:~/OSSP/Week5$ nano pipe_exec.c
chakrika@blue:~/OSSP/Week5$ gcc pipe_exec.c -o pipe_exec
chakrika@blue:~/OSSP/Week5$ ./pipe_exec
-rwxr-xr-x 1 chakrika chakrika 163444 Sep 17 13:26 pipe_exec
-rw-r--r-- 1 chakrika chakrika   883 Sep 17 13:25 pipe_exec.c
-rwxr-xr-x 1 chakrika chakrika 16360 Sep 17 13:18 producer_consumer
-rw-r--r-- 1 chakrika chakrika  1261 Sep 17 13:17 producer_consumer.c
Command pipeline completed.

```

Observation

The output of `ls -l` was passed through the pipe to `grep ".c"`, which displayed the matching C files.

Result

The shell command `ls -l | grep ".c"` was successfully implemented using system calls.